CORE-6: Database Systems - Unit-1

What is Data? Data is a collection of a distinct small unit of information. It can be used in a variety of forms like text, numbers, media, bytes, etc. it can be stored in pieces of paper or electronic memory, etc. Word 'Data' is originated from the word 'datum' that means 'single piece of information.' It is plural of the word datum.

What is Database? A database is an organized collection of data, so that it can be easily accessed and managed.

Database management system (DBMS):

- Database management system is a software which is used to manage the database. For example: MySQL, Oracle, etc are very popular commercial DBMS which are used in different applications.
- DBMS provides an interface to perform various operations like database creation, storing, retrieving, updating data and a lot more.
- It also provides monitoring users, maintaining data integrity, concurrency, protection and security to the database. In the case of multiple users, it also maintains data consistency.

Advantages of DBMS

- **Controls database redundancy:** It can control data redundancy because it stores all the data in one single database file and that recorded data is placed in the database.
- **Data sharing:** In DBMS, the authorized users of an organization can share the data among multiple users.
- **Easily Maintenance:** It can be easily maintainable due to the centralized nature of the database system.
- **Reduce time:** It reduces development time and maintenance need.
- **Backup:** It provides backup and recovery subsystems which create automatic backup of data from hardware and software failures and restores the data if required.
- **multiple user interface:** It provides different types of user interfaces like graphical user interfaces, application program interfaces

Types of Database:

1) Centralized Database: It is the type of database that stores data at a centralized database system. It comforts the users to access the stored data from different locations through several applications. These applications contain the authentication process to let users access data securely. An example of a Centralized database can be Central Library that carries a central database of each library in a college/university.

Advantages of Centralized Database

- It has decreased the risk of data management, i.e., manipulation of data will not affect the core data.

- Data consistency is maintained as it manages data in a central repository.
- It provides better data quality, which enables organizations to establish data standards.
- It is less costly because fewer vendors are required to handle the data sets.

Disadvantages of Centralized Database

- The size of the centralized database is large, which increases the response time for fetching the data.
- It is not easy to update such an extensive database system.
- If any server failure occurs, entire data will be lost, which could be a huge loss.

2) Distributed Database: Unlike a centralized database system, in distributed systems, data is distributed among different database systems of an organization. These database systems are connected via communication links. Such links help the end-users to access the data easily. **Examples** of the Distributed database are Apache Cassandra, HBase, Ignite, etc. We can further divide a distributed database system into:

- **Homogeneous DDB:** Those database systems which execute on the same operating system and use the same application process and carry the same hardware devices.
- **Heterogeneous DDB:** Those database systems which execute on different operating systems under different application procedures, and carries different hardware devices.

Advantages of Distributed Database

- Modular development is possible in a distributed database, i.e., the system can be expanded by including new computers and connecting them to the distributed system.
- One server failure will not affect the entire data set.

3) Cloud database: Cloud database facilitates us to store, manage, and retrieve data via a cloud platform. This data is accessible over the Internet. Cloud databases are also called a database as service (DBaaS) because they are offered as a managed service. Some best cloud options are:

- AWS (Amazon Web Services)
- Snowflake Computing
- Oracle Database Cloud Services
- Microsoft SQL server
- Google cloud spanner

Advantages of cloud database

Lower costs: Generally, company provider does not have to invest in databases. It can maintain and support one or more data centers.
Automated: Cloud databases are enriched with a variety of automated processes such as recovery, failover, and auto-scaling.

Increased accessibility: We can access our cloud-based database from any location, anytime. All we need is just an internet connection.

RDBMS (Relational Database Management System): The word RDBMS is termed as 'Relational Database Management System.' RDBMS is based on the Relational model, which stores data in the form of rows (tupple) and columns (attributes), and together forms a table(relation). It was introduced by E. F. Codd. A relational database uses SQL for storing, manipulating, as well as maintaining the data. An RDBMS is a tabular DBMS that maintains the security, integrity, accuracy, and consistency of the data. **Examples** of Relational databases are MySQL, Microsoft SQL Server, Oracle, etc.

Difference between DBMS and RDBMS:

DBMS	RDBMS
Normalization is not present in DBMS.	Normalization is present in RDBMS.
DBMS does not apply any integrity constraints with regards to data manipulation.	RDBMS defines the integrity constraint for the purpose of ACID property.
DBMS does not support distributed database.	RDBMS supports distributed database.
DBMS is meant to be for small organization and deal with small data. It supports single user.	RDBMS is designed to handle large amount of data. It supports multiple users.
Examples of DBMS are Fox-Pro, xml etc.	Example of RDBMS are mysql, sql server, oracle etc.

DBMS vs. File System:

DBMS	File System
DBMS is a collection of data. In DBMS, the user is not required to write the procedures.	File system is a collection of data. In this system, the user has to write the procedures for managing the database.
DBMS gives an abstract view of data that hides the details.	File system provides the detail of the data representation and storage of data.
DBMS provides a crash recovery mechanism, i.e., DBMS protects the user from the system failure.	File system doesn't have a crash mechanism, i.e., if the system crashes while entering some data, then the content of the file will be lost.
DBMS provides a good protection mechanism.	It is very difficult to protect a file under the file system.

DBMS contains a wide variety of sophisticated techniques to store and retrieve the data.	File system can't efficiently store and retrieve the data.

Database Users: Database users are the one who really use and take the benefits of database. There are different types of users depending on their need and way of accessing the database.

1. Database Administrator
2. System Analyst
3. Database Designer
4. Application Programmers
5. Naive Users / Parametric Users
6. Sophisticated Users
7. Casual Users / Temporary Users

1. **Database Administrator (DBA):** Database Administrator is a person/team who defines the schema and also controls the 3 levels of database. DBA is responsible for the following activities.
   - DBA is responsible for creating the users who wants to use the database.
   - DBA also monitors the recovery and back up and provide technical support.
   - The DBA has a DBA account in the DBMS which called a system or super user account.
   - DBA repairs damage caused due to hardware and/or software failures.
   - DBA is also responsible for providing security to the data base and he allows only the authorized users to access/modify the data base.
2. **System Analyst :** System Analyst is a user who analyzes the requirements of parametric end users. They check whether all the requirements of end users are satisfied.
3. **Data Base Designers:** Data Base Designers are the users who design the structure of data base which includes tables, indexes, views, constraints, triggers, stored procedures. He/she controls what data must be stored and how the data items to be related.
4. **Application Program :** Application Program are the back end programmers who writes the code for the application programs.
5. **Naive / Parametric End Users :** Parametric End Users are the unsophisticated who don't have any DBMS knowledge but they frequently use the data base applications in their daily life to get the desired results. For examples, Railway's ticket booking users are naive users. Clerk in any bank are naive users because they don't have any DBMS knowledge but they still use the database and perform their given task.
6. **Sophisticated Users:** Sophisticated users can be engineers, scientists, business analyst, who are familiar with the database. They can develop their own data base applications according to their requirement. They don't write the program code but they interact the data base by writing SQL queries directly through the query processor.
7. **Casual Users / Temporary Users :** Casual Users are the users who occasionally use/access the data base but each time when they access the

data base they require the new information, for example, Middle or higher level manager.

DBMS Architecture: DBMS architecture depends upon how users are connected to the database to get their request done. Database architecture is of three types like: 1-tier architecture**,** 2-tier architecture and 3-tier architecture.

1-Tier Architecture

- In this architecture, the database is directly available to the user. It means the user can directly sit on the DBMS and uses it.
- Any changes done here will directly be done on the database itself. It doesn't provide a handy tool for end users.
- The 1-Tier architecture is used for development of the local application, where programmers can directly communicate with the database for the quick response.

2-Tier Architecture

- The 2-Tier architecture is same as basic client-server. In the two-tier architecture, applications on the client end can directly communicate with the database at the server side. For this interaction, API's like: **ODBC**, **JDBC** are used.
- The user interfaces and application programs are run on the client-side.
- The server side is responsible to provide the functionalities like: query processing and transaction management.
- To communicate with the DBMS, client-side application establishes a connection with the server side.

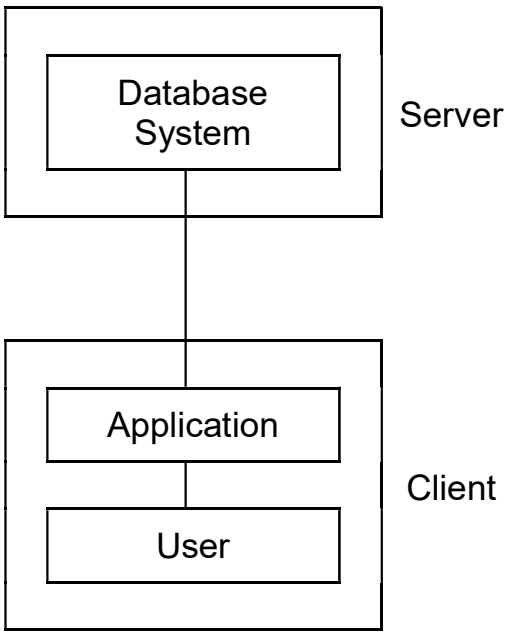


Fig: 2-tier Architecture

3-Tier Architecture

- The 3-Tier architecture contains another layer between the client and server. In this architecture, client can't directly communicate with the server.
- The application on the client-end interacts with an application server which further communicates with the database system.
- End user has no idea about the existence of the database beyond the application server. The database also has no idea about any other user beyond the application.
- The 3-Tier architecture is used in case of large web application.

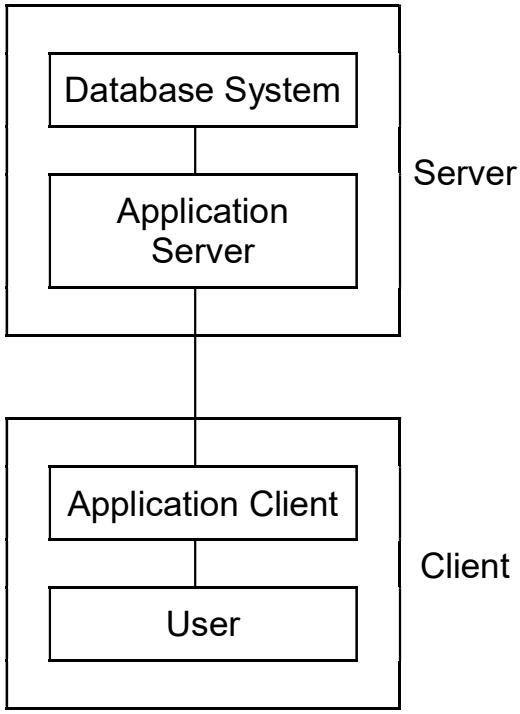
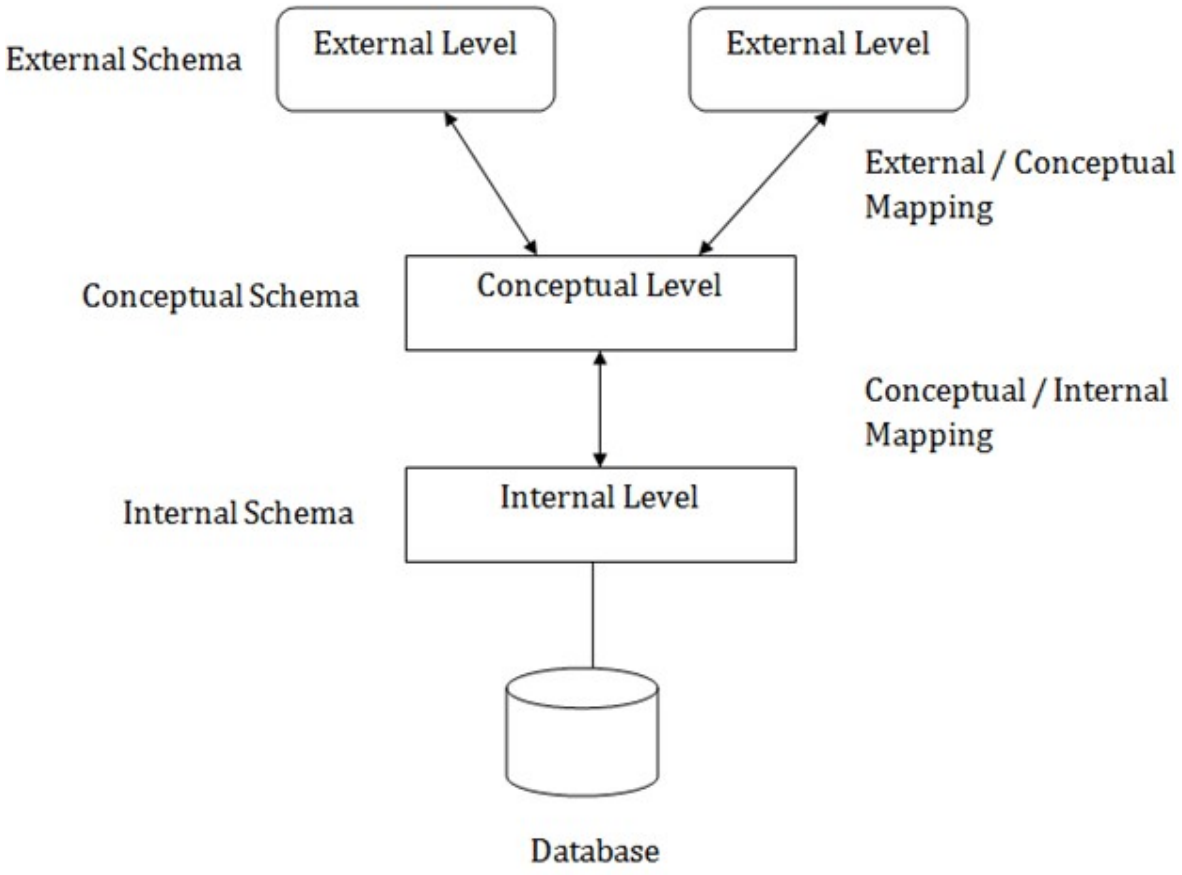


Fig: 3-tier Architecture

Three schema Architecture: The three schema architecture contains three-levels. It breaks the database down into three different categories.



1. Internal Level

- The internal level has an internal schema which describes the physical storage structure of the database.
- The internal schema is also known as a **physical schema**.
- It uses the physical data model. It is used to define that how the data will be stored in a block.
- The **physical level is used to describe complex low-level data structures** in detail.

2. Conceptual Level

- The conceptual schema describes the design of a database at the conceptual level. Conceptual level is also known as **logical level**.
- The conceptual schema describes the structure of the whole database.
- The conceptual level describes what data are to be stored in the database and also describes what relationship exists among those data.
- In the conceptual level, internal details such as an implementation of the data structure are hidden.
- **Programmers and database administrators** work at this level.

3. External Level

- At the external level, a database contains several schemas that sometimes called as **subschema**. The subschema is used to describe the different view of the database.
- An external schema is also known as **view schema**.
- Each view schema describes the database part that a particular user group is interested and hides the remaining database from that user group.
- The **view schema describes the end user interaction** with database systems.

Data Independence: Data independence is ability to modify a schema definition in one level without affecting a schema definition in the next higher level. There are two levels of data independence:

1. Physical Data Independence
2. Logical Data Independence

These are described below.

1. Logical Data Independence

- Logical data independence is ability to modify the conceptual schema without requiring any change in application programs.
- Modification at the logical levels is necessary whenever the logical structure of the database is altered.
- Logical data independence separates external level from the conceptual level.
- It is difficult to achieve logical data independence.
- Application programs are heavily dependent on logical structures of the data they access. So any change in logical structure also requires programs to change.

2. Physical Data Independence

- Physical Data Independence is the ability to modify the physical schema without requiring any change in logical schema.
- Modifications at the internal levels are occasionally necessary to improve performance. Possible modifications at internal levels are change in file structures, compression techniques, hashing algorithms, storage devices, etc.
- Physical data independence separates conceptual levels from the internal levels.
- It is easy to achieve physical data independence.

Schema and Instance

- The data which is stored in the database at a particular moment of time is called an **instance** of the database.
- The overall design of a database is called **schema**.
- A database schema is the skeleton structure of the database. It represents the logical view of the entire database.
- A schema contains schema objects like table, foreign key, primary key, views, columns, data types, stored procedure, etc.
- A database schema can be represented by using the visual diagram. That diagram shows the database objects and relationship with each other.
- A database schema is designed by the database designers to help programmers whose software will interact with the database. The process of database creation is called data **modelling**.

STUDENT

Name	Student_number	Class	Major

COURSE

Course_name	Course_number	Credit_hours	Department

PREREQUISITE

Course_number	Prerequisite_number

SECTION

Section_identifier	Course_number	Semester	Year	Instructor

GRADE_REPORT

Student_number	Section_identifier	Grade

A schema diagram can display only some aspects of a schema like the name of record type, data type, and constraints. Other aspects can't be specified through the schema diagram. For example, the given figure neither shows the data type of each data item nor the relationship among various files.

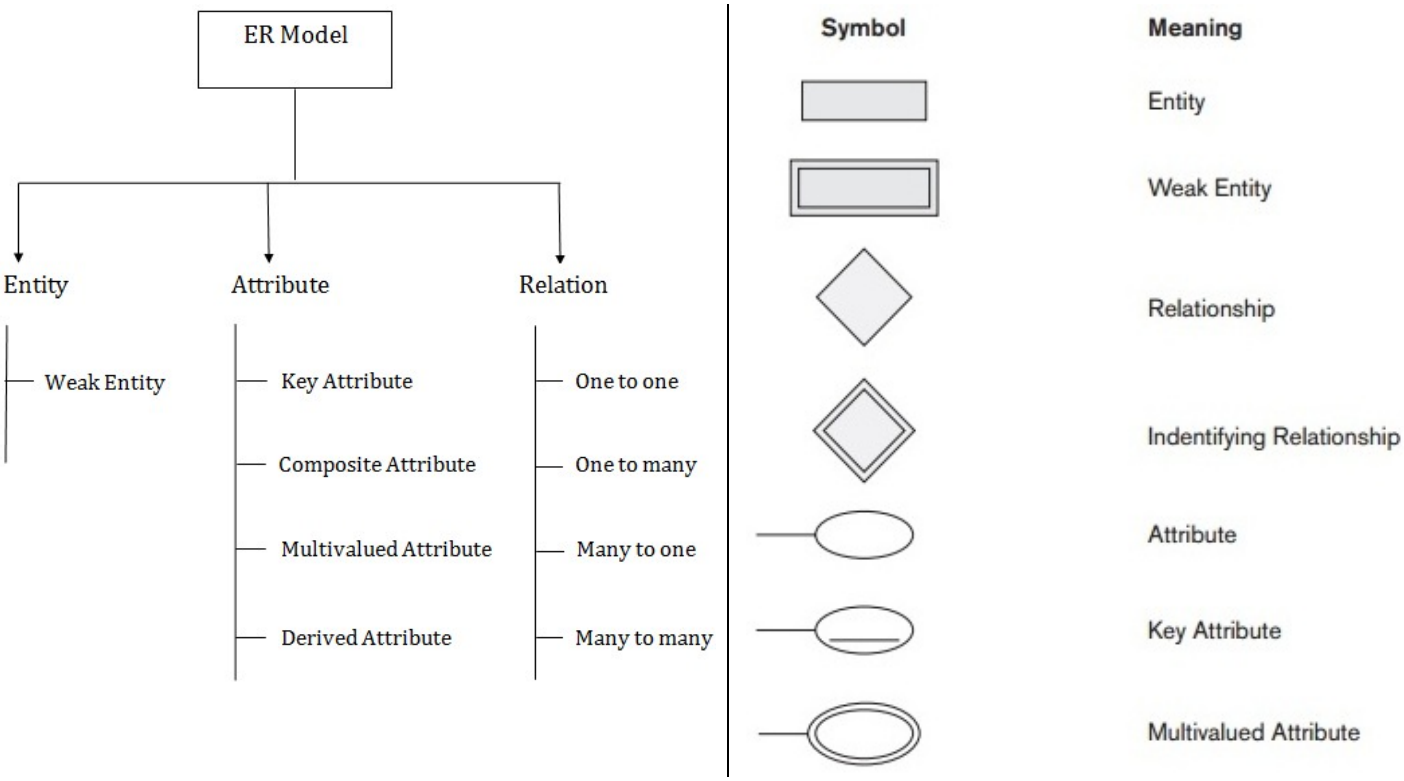
In the database, actual data changes quite frequently. For example, in the given figure, the database changes whenever we add a new grade or add a student. The data at a particular moment of time is called the instance of the database.

Data Models: A database model shows the logical structure of a database, including the relationships and constraints that determine how data can be stored and accessed. There are basically four data models:

- Hierarchical Model
- Network Model
- Entity-relationship Model
- Relational Model

ER model: ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system. It develops a conceptual design for the database. It also develops a very simple and easy to design view of data. In ER modelling, the database structure is portrayed as a diagram called an entity-relationship diagram. For example, suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.

Component of ER Diagram



1. Entity: An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as **rectangles**. Consider an organization as an example-manager, product, employee, department etc. can be taken as an entity.

a. Weak Entity: An entity that depends on another entity is called a weak entity. The weak entity doesn't contain any key attribute of its own. The weak entity is represented by a **double rectangle**.

2. Attribute: The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute. **For example,** id, age, contact number, name, etc. can be attributes of a student.

a. Key Attribute: The key attribute is the attribute which uniquely represent a tupple in an entity. It represents a primary key. The key attribute is represented by an **ellipse with the text underlined.**

b. Composite Attribute: An attribute that composed of many other attributes is known as a composite attribute. The composite attribute is represented by an ellipse, and those ellipses are connected with an ellipse.

c. Multi-valued Attribute: An attribute can have more than one value. These attributes are known as a multi-valued attribute. The **double oval** is used to represent multi-valued attribute. **For example,** a student can have more than one **phone number**.

d. Derived Attribute: An attribute that can be derived from other attribute is known as a derived attribute. It is represented by a **dashed ellipse**. **For example,** A person's age changes over time and can be derived from another attribute like Date of birth.

3. Relationship: A relationship is used to describe the relation between entities. **Diamond or rhombus** is used to represent the relationship. There can be four Types of relationship as follows:

a. One-to-One Relationship: When only one instance of an entity is associated with the relationship, then it is known as one to one relationship. **For example,** A female can marry to one male, and a male can marry to one female.

b. One-to-many relationship: When only one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then this is known as a one-to-many relationship. **For example,** Scientist can invent many inventions, but the invention is done by the only specific scientist.

c. Many-to-one relationship: When more than one instance of the entity on the left, and only one instance of an entity on the right associates with the relationship then it is known as a many-to-one relationship. **For example,** Student enrolls for only one course, but a course can have many students.

d. Many-to-many relationship: When more than one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then it is known as a many-to-many relationship. **For**

example, Employee can assign by many projects and project can have many employees.

4. Keys: Key is an attribute which uniquely identify a record or row of data from the table. It is also used to establish and identify relationships between tables. **For example,** In Student table, ID is used as a key because it is unique for each student. There are mainly four types of keys.

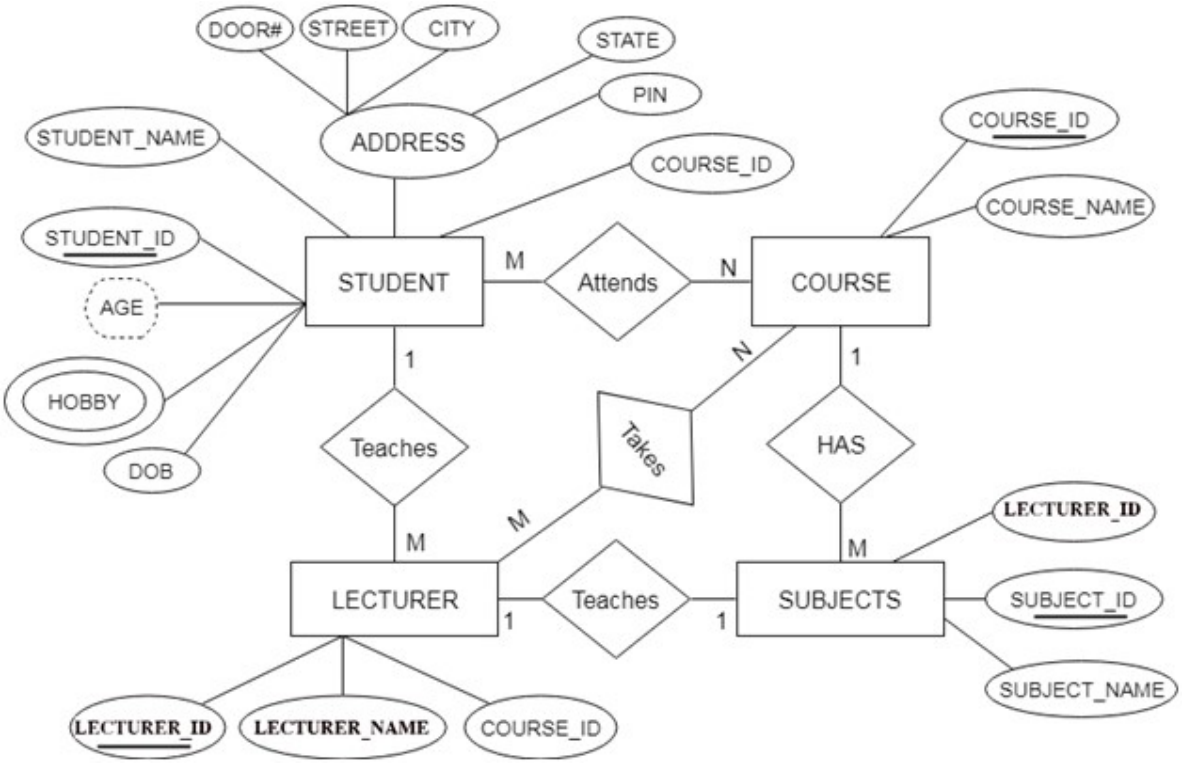
i. Super Key: Super key is a set of attribute which can uniquely identify a tupple. **For example,** In EMPLOYEE table, name of two employees can be the same, but their EMPLYEE_ID can't be the same. Hence, the combination of (EMPLOYEE_ID, EMPLOYEE-NAME) can also be a key. This key is a super key.

ii. Candidate key: The minimal super key is known as candidate key. **For example,** In the EMPLOYEE table: id, SSN, Passport_Number, and License_Number, etc. are considered as a candidate key.

iii. Primary key: One of the candidate key which is most suitable is chosen as Primary key. **For Example**, EMPLOYEE table contains multiple keys like id, SSN, Passport_Number, and License_Number, etc. The key, id can be chosen as primary key. Rest of the attributes like SSN, Passport_Number, and License_Number, etc. are considered as a candidate key. For each entity, selection of the primary key is based on requirement and developers.

iv. Foreign key: Foreign keys are the column of the table which is used to point to the primary key of another table. In a company, every employee works in a specific department, and employee and department are two different entities. So we can't store the information of the department in the employee table. That's why we link these two tables through the primary key of one table. We add the primary key of the DEPARTMENT table, Department_Id as a new attribute in the EMPLOYEE table. Now in the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.

Reduction of ER diagram to Table: Let us consider the following ER diagram

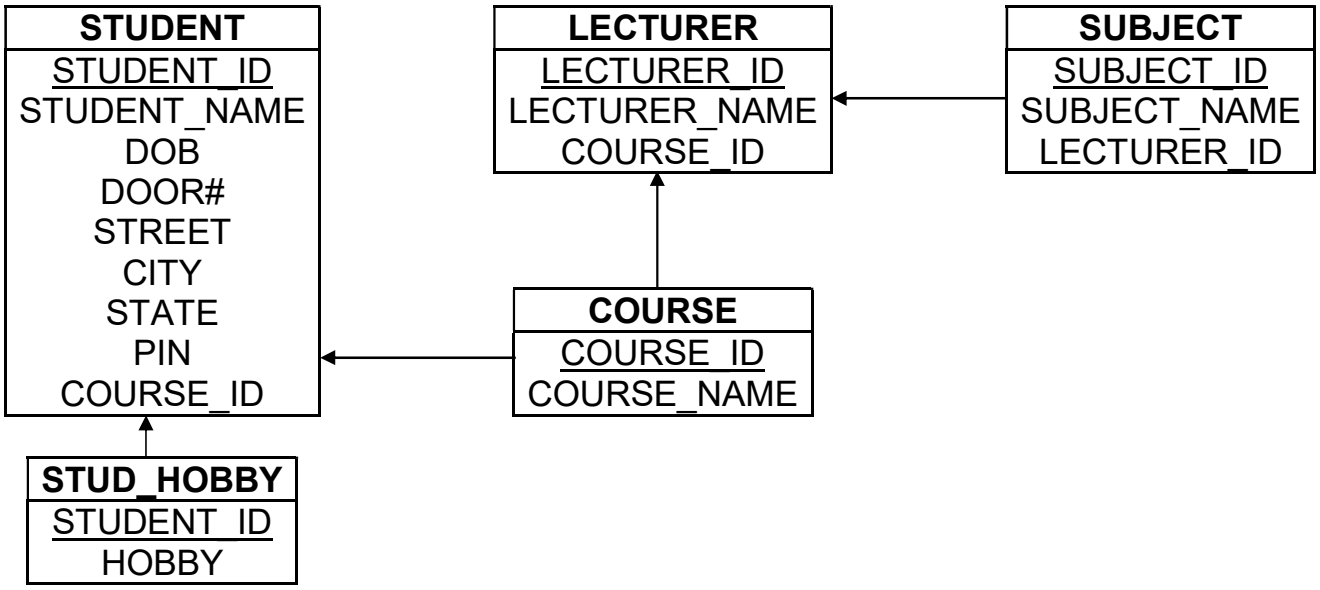


There are some points for converting the ER diagram to the table:

- **Entity type becomes a table:** In the given ER diagram, LECTURE, STUDENT, SUBJECT and COURSE forms individual tables.
- **All single-valued attribute becomes a column for the table:** In the STUDENT entity, STUDENT_NAME and STUDENT_ID form the column of STUDENT table. Similarly, COURSE_NAME and COURSE_ID form the column of COURSE table and so on.
- **A key attribute of the entity type represented by the primary key:** In the given ER diagram, COURSE_ID, STUDENT_ID, SUBJECT_ID, and LECTURE_ID are the key attribute of the entity.
- **The multivalued attribute is represented by a separate table:** In the student table, a hobby is a multivalued attribute. So it is not possible to represent multiple values in a single column of STUDENT table. Hence we create a table STUD_HOBBY with column name STUDENT_ID and HOBBY. Using both the column, we create a composite key.
- **Composite attribute represented by components:** In the given ER diagram, student address is a composite attribute. It contains CITY, PIN, DOOR#, STREET, and STATE. In the STUDENT table, these attributes can merge as an individual column.

- **Derived attributes are not considered in the table:** In the STUDENT table, Age is the derived attribute. It can be calculated at any point of time by calculating the difference between current date and Date of Birth.

Using these rules, we can convert the ER diagram to tables and columns and assign the mapping between the tables. Table structure for the given ER diagram is as follows:



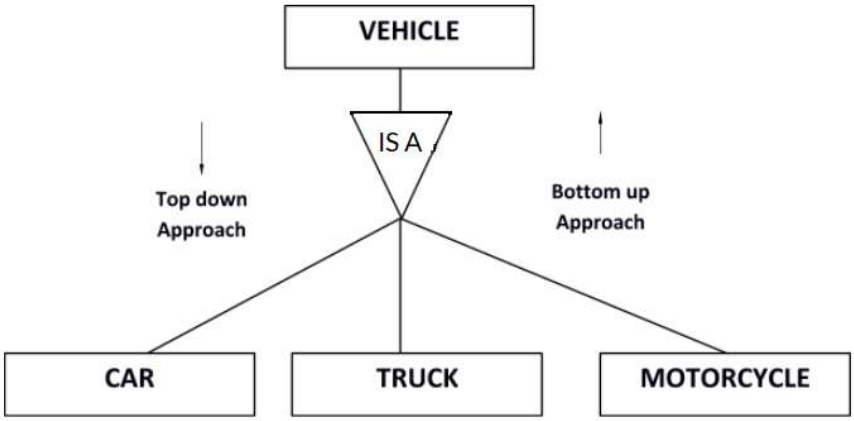
ER Naming Conventions:

- In our ER diagrams, we will use the convention that entity type and relationship type names are uppercase letters.
- Attribute names have their initial letter capitalized, and role names are lowercase letters.
- Name of the relation is distinct from all other relations.
- Each attribute contains a distinct name

Enhanced Entity-Relationship (EER) Model: The enhanced entity–relationship **(**EER**)** model (or extended entity–relationship model) is a high-level or conceptual data model incorporating extensions to the original entity–relationship (ER) model, used in the design of databases. It was developed to reflect more precisely the properties and constraints that are found in more complex databases. The EER model includes all of the concepts introduced by the ER model. Additionally it includes the concepts of a subclass and superclass (IS-A), along with the concepts of specialization and generalization. Furthermore, it introduces the concept of a union type or category, which is used to represent a collection of objects that is the union of objects of different entity types. EER model also includes EER diagrams that are conceptual models that accurately represent the requirements of complex databases.

Subclass and super-class: Entity type Y is a subtype (subclass) of an entity type X if and only if every Y is necessarily an X. A subclass entity inherits all attributes and relationships of its super-class entity. This property is called the attribute and relationship inheritance. A subclass entity may have its own specific attributes and relationships (together with all the attributes and relationships it inherits from the super-class). A common super-class example is a Vehicle super-class along with the subclasses of Car and Truck. There are a number of common attributes between a car and a truck, which would be part of the super-class, while the attributes specific to a car or a truck (such as max payload, truck type...) would make up two subclass.

Specialization and Generalization:



Generalization is a process of generalizing an entity which contains generalized attributes or properties of generalized entities. It is a Bottom up process i.e. consider we have 3 sub entities Car, Truck and Motorcycle. Now these three entities can be generalized into one super class named as Vehicle.

Specialization is a process of identifying subsets of an entity that share some different characteristic. It is a top down approach in which one entity is broken down into low level entity. In above example Vehicle entity can be a Car, Truck or Motorcycle.

Functional Dependencies: The functional dependency is a relationship that exists between two attributes. It typically exists between the primary key and non-key attribute within a table. If X and Y are two attributes of a relation and Y is functionally dependant on X then it is represent as: **X → Y**

The left side of FD is known as a determinant, the right side of the production is known as a dependent. **For example,** Assume we have an employee table with attributes Emp_Id, Emp_Name, Emp_Address. Here Emp_Id attribute can uniquely identify the Emp_Name attribute of employee table because if we know the Emp_Id, we can tell that employee name associated with it. Functional dependency can be written as: **Emp_Id → Emp_Name**.

We can say that Emp_Name is functionally dependent on Emp_Id.

Types of Functional dependency

1. Trivial functional dependency

- A → B has trivial functional dependency if B is a subset of A.
- The following dependencies are also trivial like: A → A, B → B

Example:

Consider a table with two columns Employee_Id and Employee_Name.

{Employee_id, Employee_Name} → Employee_Id is a trivial functional dependency as Employee_Id is a subset of {Employee_Id, Employee_Name}. Also, Employee_Id → Employee_Id and Employee_Name →Employee_Name are trivial dependencies too.

2. Non-trivial functional dependency

- A → B is a non-trivial functional dependency if B is not a subset of A.
- When A intersection B is NULL, then A → B is called as complete non-trivial.

Example: ID → Name

Normalization

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations.
- It is also used to eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization divides the larger table into the smaller table and links them using relationship.

There are six level of normalization.

1. First Normal Form (1NF)
2. Second Normal Form (2NF)
3. Third Normal Form (3NF)
4. Boyce-Codd Normal Form (BCNF)
5. Forth Normal Form (4NF)
6. Fifth Normal Form (5NF)

First Normal Form (1NF): A relation is in 1NF if there are only **Single Valued** Attributes or **atomic attribute**. In other words the table should not contain multi-valued attribute, composite attribute or their combinations.

Example:

CUSTOMER		
Customer ID	**Name**	**Telephone Number**
123	P kar	9937147258, 9937369258
456	Raj	9583852741, 8847963852
789	John	8038456987

(Customer relation is not in 1NF)

CUSTOMER			
Customer ID	**Name**	**Telephone Number1**	**Telephone Number2**
123	P Kar	9937147258	9937369258
456	Raj	9583852741	8847963852
789	John	8038456987	

(Now Customer relation is in 1NF)

Second Normal Form (2NF): A relation is in 2NF if it satisfies the following conditions:

- The relation must be in 1NF.
- It has **no Partial Dependency**, i.e., no non-prime attribute (attributes which are not part of any candidate key) is dependent on any proper subset of any candidate key of the table. In other words every non-prime attribute is fully functional dependent on the primary key

Example:

STUDENT table		
STUD_ NO	**COURSE _NO**	**COURSE_ FEE**
1	C1	1000
2	C2	1500
1	C4	2000
4	C3	1000
4	C1	1000
2	C5	2000

(NOT in 2NF)

STUD_COURSE table	
STUD_ NO	**COURSE _NO**
1	C1
2	C2
1	C4
4	C3
4	C1
2	C5

COURSE table	
COURSE _NO	**COURSE_ FEE**
C1	1000
C2	1500
C3	1000
C4	2000
C5	2000

(Decomposed into 2NF)

In the above example, (STUD_NO, COURSE_NO) is the candidate key in STUDENT table. So the prime attributes are STUD_NO and COURSE_NO and the only not prime attribute is COURSE_FEE . The functional dependencies are:

(STUD_NO, COURSE_NO) → COURSE_FEE
COURSE_NO → COURSE_FEE

Here, COURSE_FEE which is a non-prime attribute is partially dependent on the key attribute. So the STUDENT table is not in 2NF. It is decomposed into two tables STUD_COURSE table and COURSE table which are in 2NF.

Third Normal Form (3NF): A relation will be in 3NF if it is in 2NF and does not contain **any transitive dependency.**

OR

A relation is in 3NF if at least one of the following conditions holds for every non-trivial function dependency **X → Y:**

1. X is a super key **or**
2. Y is a prime attribute (each element of Y is part of some candidate key).

Example:

EMPLOYEE_DETAIL table:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harry	201010	UP	Noida
333	Stephan	02228	US	Boston
444	Lan	60007	US	Chicago
555	Katharine	06389	UK	Norwich
666	John	462007	MP	Bhopal

Super key in the above table:
{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}...so on

Candidate key: {EMP_ID}

Non-prime attributes: All attributes except EMP_ID are non-prime.

FDs: EMP_ZIP → { EMP_STATE, EMP_CITY }
EMP_ID → EMP_ZIP

So, the non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key (EMP_ID). It violates the rule of third normal form.

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMPLOYEE table

EMP_ID	EMP_NAME	EMP_ZIP
222	Harry	201010
333	Stephan	202228
444	Lan	260007
555	Katharine	206389
666	John	462007

EMPLOYEE_ZIP table

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
202228	US	Boston
260007	US	Chicago
206389	UK	Norwich
462007	MP	Bhopal

(EMPLOYEE_DETAIL table broken into two tables EMPLOYEE table and EMPLOYEE_ZIP table)

Boyce Codd normal form (BCNF): A table is in BCNF if it in 3NF and every functional dependency **X → Y, X is the super key** of the table. So, BCNF is stricter than 3NF.

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE table

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table, Functional dependencies are as follows:

EMP_ID → EMP_COUNTRY

EMP_DEPT → { DEPT_TYPE, EMP_DEPT_NO }

Candidate key: { EMP-ID, EMP-DEPT }

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it into three tables:

EMP_COUNTRY table

EMP_ID	EMP_COUNTRY
264	India
264	India

EMP_DEPT table

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING table

EMP_ID	EMP_DEPT
D394	283
D394	300
D283	232
D283	549

Functional dependencies:

EMP_ID → EMP_COUNTRY
EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate keys:

For the first table (EMP_COUNTRY table): EMP_ID
For the second table(EMP_DEPT table): EMP_DEPT
For the third table(EMP_DEPT_MAPPING table): {EMP_ID, EMP_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.

Fourth normal form (4NF): A relation R is in 4NF if and only if the following conditions are satisfied:

1. It should be in the Boyce-Codd Normal Form (BCNF).
2. The table should not have any Multi-valued Dependency.

Multivalued Dependency

- Multivalued dependency occurs when two attributes in a table are independent of each other but, both depend on a third attribute.
- A multivalued dependency consists of at least two attributes that are dependent on a third attribute that's why it always requires at least three attributes.

Example:

UNIVERSITY_COURSES

COURSE	BOOK	LECTURER
DBMS	Korth	John
DBMS	Desai	John
DBMS	Korth	William
DBMS	Desai	William

(NOT in 4NF)

COURSE_ BOOK

COURSE	BOOK
DBMS	Korth
DBMS	Desai

COURSE_ LECT

COURSE	LECTURER
DBMS	John
DBMS	William

(UNIVERSITY_COURSES table decomposed into COURSE_ BOOK table and COURSE_ LECT TABLE to make 4NF)

Consider the example of a relation of UNIVERSITY_COURSES, the books recommended for the course, and the lecturers who will be teaching the course:

Because the lecturers attached to the course and the books attached to the course are independent of each other, this database design has a multivalued dependency.

COURSE →→ BOOK

COURSE →→ LECTURER

Thus, databases with multivalued dependencies exhibit redundancy. In database normalization, fourth normal form requires that for every nontrivial multivalued dependency X →→Y, X is a superkey.

Fifth normal form (5NF):

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

Example:

In this table, John takes both Computer and Math class for Semester 1 but he doesn't take Math class for Semester 2. In this case, combination of all these fields required to identify a valid data. Suppose we add a new Semester as Semester 3 but do not know about the subject and who will be taking that subject so we leave Lecturer and Subject as NULL. But all three columns together acts as a primary key, so we can't leave other two columns blank.

SUBJECT	LECTURER	SEMESTER
Computer	Anshika	Semester 1
Computer	John	Semester 1
Math	John	Semester 1
Math	Akash	Semester 2
Chemistry	Praveen	Semester 1

So to make the above table into 5NF, we can decompose it into three relations P1, P2 & P3:

P1

SEMESTER	SUBJECT
Semester 1	Computer
Semester 1	Math
Semester 1	Chemistry
Semester 2	Math

P2

SUBJECT	LECTURER
Computer	Anshika
Computer	John
Math	John
Math	Akash
Chemistry	Praveen

P3

SEMSTER	LECTURER
Semester 1	Anshika
Semester 1	John
Semester 1	John
Semester 2	Akash
Semester 1	Praveen